of the Exploitation phase and suggests that further refinement is necessary to address the intricacies of this critical stage. Nevertheless, by strategically delaying the automation of penetration testing to later stages, VulnBot ensures that critical subtasks are executed with greater precision, thereby increasing the likelihood of completing the testing process.

5.2 Ablation Study (RQ2)

In this section, we evaluate the impact of key architectural components by conducting ablation experiments on AUTOPENBENCH Real-world tasks. The experiments focus on the Llama3.1-405B model within a 128k token context. We implement three variants of VulnBot to isolate the contributions of its core components: (1) VulnBot-without Role: The role specialization mechanism is deactivated, causing agents to operate without distinct roles. (2) VulnBot-without PTG: The Penetration Task Graph (PTG) is removed, eliminating the structured task planning and dependency management. (3) VulnBot-without Summarizer: The Summarizer module is disabled, preventing inter-agent communication and context summarization.

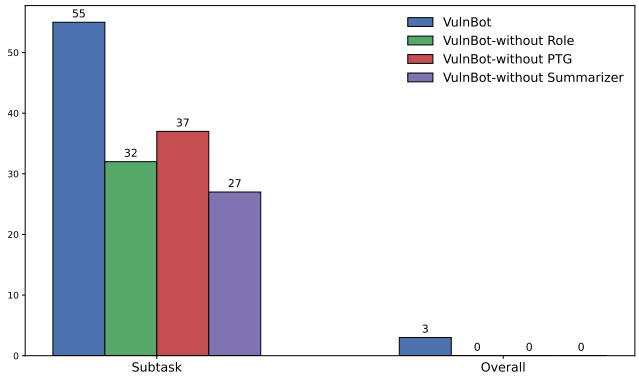

Figure 5: Ablation study of VulnBot on AUTOPENBENCH. This figure demonstrates the impact of removing key components—role specialization, the PTG, and the Summarizer—on model performance.

Figure 5 illustrates the performance degradation observed when these essential components are removed. Our findings reveal that each component plays a critical role in enhancing model performance. Specifically, the removal of role specialization results in a significant decline in performance, with subtask success rates dropping from 55 to 32. Similarly, omitting the PTG leads to a reduction in the subtask success rate, decreasing to 37. The most substantial performance decline occurs when the Summarizer is removed, reducing the subtask success rate to just 27. Furthermore, the overall task success rate is entirely eliminated when any of these components are removed. These results underscore the critical importance of role specialization, PTG, and the Summarizer in achieving high performance on penetration testing tasks. The ablation study highlights that the synergistic interaction of these components is vital for the model's success in both subtasks and overall task completion. This finding aligns with the broader trend in multi-agent systems, where effective role allocation, task planning, and communication are essential for complex, real-world applications.

5.3 Effectiveness for Real-World (RQ3)

To evaluate the practical applicability of our models, we conducted five rounds of experiments on a selection of real-world targets from the AI-Pentest-Benchmark, which includes 13 vulnerable machines. We selected six machines for this evaluation, focusing on penetration tasks that did not involve image observation or human intervention. The experiments were conducted using two models: Llama3.1-405B with a 128k context and DeepSeek-v3 with a 64k context. The task completion rates were calculated based on the successful completion of subtasks as defined by the AI-Pentest-Benchmark. For each machine, the reported completion rate represents the best performance achieved across the five experimental runs. Figure 6 illustrates the subtask completion rates across these machines, with a value of 1 indicating a successful penetration. The results demonstrate that VulnBot-Llama3.1-405B consistently outperforms its counterparts, achieving the highest completion rates on Victim1 (0.33), Library2 (0.40), and WestWild (0.57). Similarly, VulnBot-DeepSeek-v3 demonstrated competitive performance, with completion rates of 0.83 on Victim1 and 0.71 on WestWild. These findings highlight VulnBot's superior capability in handling complex, multi-step attack chains, which are critical in real-world penetration testing scenarios. The consistent performance of VulnBot across diverse machines underscores its robustness and adaptability, making it a reliable tool for practical cybersecurity applications.

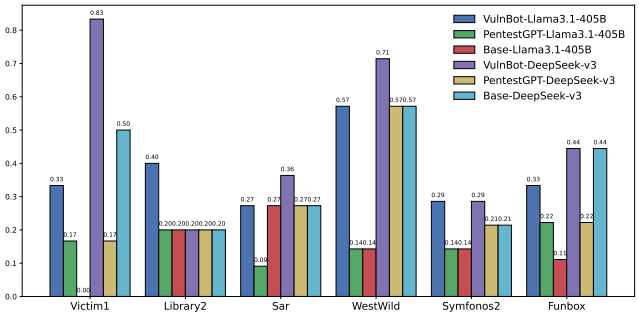

Figure 6: The performance of VulnBot over the real-world machines.

5.4 Retrieval Augmented Generation (RQ4)

To further investigate whether prior penetration knowledge can enhance the performance of our framework, we integrated the Memory Retriever module into the Llama3.1-405B model, which supports a 128k context window. This integration leverages RAG to improve the model's contextual understanding and task-specific optimization. In this experiment, we evaluated the performance of three distinct systems: Llama3.1-405B with RAG, GPT-4o with Manual, and Llama3.1-405B with Manual. The data for GPT-4o and Llama3.1 with Manual were obtained from [33], where human operators utilized PentestGPT tools. To augment the contextual knowledge of native LLMs, we incorporated content from cybersecurity resources such as HackTricks [26] and HackingArticles [6]. This content was segmented into 750-word chunks, and the resulting embeddings were stored in the Milvus vector database [40] for efficient retrieval. This approach enables the system to dynamically retrieve relevant historical data and prior knowledge, thereby mitigating the hallucination problem often encountered with LLMs.

Figure 7 illustrates the task completion rates of these models across six real-world machines. The results demonstrate that integrating the Memory Retriever module significantly enhances performance on specific machines, particularly Victim1 and WestWild. Notably, VulnBot successfully executed an end-to-end penetration of the WestWild machine, showcasing its ability to complete complex tasks autonomously. These findings highlight the advantages of retrieval-augmented approaches in improving the contextual understanding and task-specific optimization of penetration testing models. The integration of the Memory Retriever module not only enhances the model's ability to retrieve and utilize relevant information but also improves its overall performance in real-world penetration testing scenarios, achieving performance comparable to or even surpassing that of human operators.

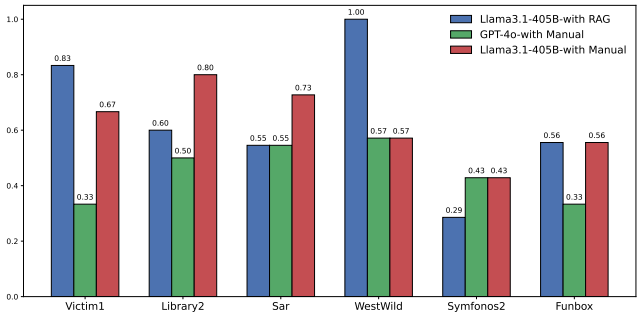

Figure 7: Performance comparison of VulnBot with Memory Retriever module

6 Discussion

The results obtained highlight VulnBot's potential for efficient vulnerability detection and exploitation. However, our findings also reveal several challenges and areas for future research that need to be addressed to enhance its capabilities further.

6.1 Limitations in Processing Non-Textual Information

A significant limitation of VulnBot is its inability to process non-textual information, such as images or graphical interfaces generated by penetration testing tools. In real-world penetration testing scenarios, such is often critical for understanding attack surfaces and interpreting the results of security scans. Currently, VulnBot depends on manual descriptions to interpret these non-textual elements, which introduces a bottleneck in achieving full automation of the penetration testing process. Future iterations of VulnBot could address this limitation by incorporating image recognition and processing capabilities. This enhancement would enable the system to analyze and extract relevant information from screenshots and other graphical representations.

6.2 Real-World Performance and Challenges

The real-world tasks in the AUTOPENBENCH include two CVEs from 2024. VulnBot completed one of these tasks using Llama3.3 and Llama3.1, despite both models having a knowledge cutoff in December 2023. This achievement highlights the reliability of our method, as it does not rely on prior knowledge of the vulnerabilities. Despite these promising results in simulated environments, completing end-to-end penetration testing on real-world machines remains a significant challenge for VulnBot. Even with RAG to enhance the model's contextual knowledge and task-specific optimizations, VulnBot still faces difficulties in achieving full autonomy and success across all stages of a real-world penetration test. These challenges stem from the complexity of real-world systems, the dynamic nature of security vulnerabilities, and the need for precise execution of multi-step attack chains.

7 Related Work

7.1 Vulnerability Detection and Exploitation

Atropos introduces a novel fuzzing technique for detecting server-side vulnerabilities in PHP-based web applications. It utilizes snapshot-based, feedback-driven fuzzing, which is integrated directly with the PHP interpreter [25]. Similarly, NAUTILUS focuses on identifying vulnerabilities in RESTful APIs through guided testing and parameter generation strategies, emphasizing complex API interactions and

corner cases [15]. Furthermore, empirical studies, such as "Understanding Hackers' Work", provide insights into the operational methods and challenges faced by offensive security practitioners, underscoring the need for improved tooling [28].

Recent advancements in large language models (LLMs) have shown significant promise in vulnerability management. Liu et al. explore the use of ChatGPT for handling complex cybersecurity tasks, particularly bug report analysis. They introduce a self-heuristic prompt template that enhances ChatGPT's performance by summarizing domain knowledge from provided examples. This approach enables the model to learn task-specific characteristics, resulting in improved performance in bug report summarization through few-shot learning compared to zero-shot or general information prompts [36]. Fang et al. further investigate LLMs' potential in exploiting one-day vulnerabilities, demonstrating that GPT-4 can autonomously exploit 87% of a benchmark set of real-world vulnerabilities when provided with CVE descriptions. This research highlights the emergent capabilities of advanced LLMs in cybersecurity, while also raising concerns about their responsible deployment [19].

7.2 Automated Penetration Testing

Recent research has explored integrating LLMs into penetration testing workflows, significantly enhancing automation and efficiency. Al-Sinani and Mitchel investigate the use of GPT-4 across various ethical hacking phases, including reconnaissance and post-exploitation [3]. Tools like AUTOATTACKER [56] and BreachSeek [4] leverage LLMs to automate post-breach activities and simulate cyberattacks, respectively, while CIPHER [48] specializes in assisting ethical researchers through structured augmentation methods. Frameworks such as PTGroup [55] and HPTSA [20] demonstrate the potential of multi-agent systems and hierarchical planning in exploiting zero-day vulnerabilities. Furthermore, HackSynth [41] and Pentest Copilot [23] highlight the role of crafted prompts and LLM integration in automating penetration testing sub-tasks.

Happe and Cito further explore the application of LLMs in penetration testing, presenting use cases for both high-level task planning and low-level vulnerability hunting. Their work implements a feedback loop where LLM-generated actions are executed on a virtual machine via SSH, demonstrating the potential of LLMs to automate parts of penetration testing while raising ethical concerns about misuse [27]. Happe et al. also evaluated the ability of different LLMs to execute privilege escalation attacks in the simulation environment by developing a fully automatic tool Wintermute. The results show that GPT-4-turbo has achieved a remarkable success rate with the assistance of sufficient context information and state mechanism [29]. Huang and Zhu introduce PenHeal, a two-stage LLM framework for automated penetration and optimal remediation. Their system incorporates components like Planner, Executor, Estimator, Advisor, and Evaluator to streamline the process, using counterfactual prompting and a Group Knapsack Algorithm to prioritize effective and cost-efficient remediations [31].

However, these approaches face limitations, including dependency on detailed vulnerability descriptions (e.g., CVE data) for effective exploitation, instability and variability in performance across different tasks and environments, and the need for human intervention in complex or end-to-end penetration testing scenarios. Additionally, many systems struggle with long-term planning and adaptability in dynamic environments, and while assisted or multi-agent approaches improve success rates, fully autonomous agents still face challenges in achieving consistent and reliable results.

7.3 Application of LLM in Cybersecurity

Beyond penetration testing, LLMs are increasingly applied to a broader range of cybersecurity tasks. Cycle enhances code generation capabilities through iterative self-refinement [16], while Guan et al. leverage LLMs to detect model optimization bugs in deep learning libraries [24]. Fang et al. demonstrate the ability of LLMs to exploit recently disclosed vulnerabilities with high success rates. Tools like SecurityBot [57] integrate LLMs with reinforcement learning to improve cybersecurity operations, and AURORA automates the orchestration of APT attack campaigns [52]. Additionally, PTHelper [12] streamlines penetration testing by integrating AI with state-of-the-art tools. These applications illustrate the versatility of LLMs in addressing diverse cybersecurity challenges.

8 Conclusion

In this paper, we present VulnBot, an autonomous penetration testing framework that LLMs and multi-agent systems. VulnBot is designed to emulate the collaborative workflows of human penetration testing teams, thereby addressing the inefficiencies and manual dependencies inherent in traditional penetration testing methodologies. By decomposing complex tasks into specialized phases—reconnaissance, scanning, and exploitation—and utilizing a PTG to ensure logical task execution, VulnBot demonstrates significant advancements in automating penetration testing workflows.

Our experimental results underscore VulnBot's superior performance relative to baseline models such as GPT-4 and Llama3. Incorporating RAG further augments VulnBot's capabilities, enabling it to execute end-to-end penetration tasks autonomously without human intervention. These findings highlight the potential of VulnBot to revolutionize the field of penetration testing by enhancing efficiency, scalability, and autonomy.

References

[1] U.s. department of the interior, 2024. https://www.doi.gov/ocio/customers/penetration-testing.

[2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[3] Haitham S Al-Sinani and Chris J Mitchell. Ai-augmented ethical hacking: A practical examination of manual exploitation and privilege escalation in linux environments. *arXiv preprint arXiv:2411.17539*, 2024.

[4] Ibrahim Alshehri, Adnan Alshehri, Abdulrahman Almalki, Majed Bamardouf, and Alaqsa Akbar. Breachseek: A multi-agent automated penetration tester. *arXiv preprint arXiv:2409.03789*, 2024.

[5] Brad Arkin, Scott Stender, and Gary McGraw. Software penetration testing. *IEEE Security & Privacy*, 3(1):84–87, 2005.

[6] Hacking Articles. Hacking articles, 2024. https://www.hackingarticles.in/.

[7] Saumick Basu. 7 penetration testing phases explained: Ultimate guide, 2024. https://www.strikegraph.com/blog/pen-testing-phases-steps.

[8] Rob Behnke. 5 phases of ethical hacking, 2021. https://www.halborn.com/blog/post/5-phases-of-ethical-hacking.

[9] Matt Bishop. About penetration testing. *IEEE Security & Privacy*, 5(6):84–87, 2007.

[10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[11] The Cyphere. Penetration testing statistics, vulnerabilities and trends in 2024, 2024. https://thecyphere.com/blog/penetration-testing-statistics/.

[12] Jacobo Casado de Gracia and Alfonso Sánchez-Macián. Pthelper: An open source tool to support the penetration testing process. *arXiv preprint arXiv:2406.08242*, 2024.

[13] Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. Masterkey: Automated jailbreaking of large language model chatbots. In *Proc. ISOC NDSS*, 2024.

[14] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. {PentestGPT}: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864, 2024.

[15] Gelei Deng, Zhiyi Zhang, Yuekang Li, Yi Liu, Tianwei Zhang, Yang Liu, Guo Yu, and Dongjin Wang. {NAUTILUS}: Automated {RESTful}{API} vulnerability detection. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 5593–5609, 2023.

[16] Yangruibo Ding, Marcus J Min, Gail Kaiser, and Baishakhi Ray. Cycle: Learning to self-refine the code generation. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):392–418, 2024.

[17] Dirb. Dirb, 2024. https://dirb.sourceforge.net/.

[18] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[19] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. Llm agents can autonomously exploit one-day vulnerabilities. *arXiv preprint arXiv:2404.08144*, 2024.

[20] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. Teams of llm agents can exploit zero-day vulnerabilities. *arXiv preprint arXiv:2406.01637*, 2024.

[21] Areej Fatima, Tahir Abbas Khan, Tamer Mohamed Abdellatif, Sidra Zulfiqar, Muhammad Asif, Waseem Safi, Hussam Al Hamadi, and Amer Hani Al-Kassem. Impact and research challenges of penetrating testing and vulnerability assessment on network threat. In *2023 International Conference on Business Analytics for Technology and Security (ICBATS)*, pages 1–8. IEEE, 2023.

[22] Luca Gioacchini, Marco Mellia, Idilio Drago, Alexander Delsanto, Giuseppe Siracusano, and Roberto Bifulco. Autopenbench: Benchmarking generative agents for penetration testing. *arXiv preprint arXiv:2410.03225*, 2024.

[23] Dhruva Goyal, Sitaraman Subramanian, and Aditya Peela. Hacking, the lazy way: Llm augmented pentesting. *arXiv preprint arXiv:2409.09493*, 2024.

[24] Hao Guan, Guangdong Bai, and Yepang Liu. Large language models can connect the dots: Exploring model optimization bugs with domain knowledge-aware prompts. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1579–1591, 2024.

[25] Emre Güler, Sergej Schumilo, Moritz Schloegel, Nils Bars, Philipp Görz, Xinyi Xu, Cemal Kaygusuz, and Thorsten Holz. Atropos: Effective fuzzing of web applications for server-side vulnerabilities. In *USENIX Security Symposium*, 2024.

[26] Hacktricks. Hacktricks, 2024. https://book.hacktricks.wiki/en/index.html.

[27] Andreas Happe and Jürgen Cito. Getting pwn'd by ai: Penetration testing with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 2082–2086, 2023.

[28] Andreas Happe and Jürgen Cito. Understanding hackers' work: An empirical study of offensive security practitioners. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1669–1680, 2023.

[29] Andreas Happe, Aaron Kaplan, and Juergen Cito. Llms as hackers: Autonomous linux privilege escalation attacks. *arXiv preprint arXiv:2310.11409*, 2024.

[30] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

[31] Junjie Huang and Quanyan Zhu. Penheal: A two-stage llm framework for automated pentesting and optimal remediation. In *Proceedings of the Workshop on Autonomous Cybersecurity*, pages 11–22, 2023.

[32] Hydra. Hydra is a game launcher with its own embedded bittorrent client, 2024. https://github.com/hydralauncher/hydra.

[33] Isamu Isozaki, Manil Shrestha, Rick Console, and Edward Kim. Towards automated penetration testing: Introducing llm benchmark, analysis, and improvements. *arXiv preprint arXiv:2410.17141*, 2024.

[34] Kali. The most advanced penetration testing distribution, 2024. https://www.kali.org/.

[35] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[36] Peiyu Liu, Junming Liu, Lirong Fu, Kangjie Lu, Yifan Xia, Xuhong Zhang, Wenzhi Chen, Haiqin Weng, Shouling Ji, and Wenhai Wang. Exploring {ChatGPT's} capabilities on vulnerability management. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 811–828, 2024.

[37] Qian Liu, Jinke Song, Zhiguo Huang, Yuxuan Zhang, Glide-The, and Liunux4odoo. langchain-chatchat, 2013. https://github.com/chatchat-space/Langchain-Chatchat.

[38] Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, Kailong Wang, and Yang Liu. Jailbreaking chatgpt via prompt engineering: An empirical study. *arXiv preprint arXiv:2305.13860*, 2023.

[39] Metasploit. Metasploit | penetration testing software, pen testing security | metasploit, 2024. https://www.metasploit.com/.

[40] Milvus. Milvus, 2024. https://milvus.io/docs/zh/quickstart.md.

[41] Lajos Muzsai, David Imolai, and András Lukács. Hacksynth: Llm agent and evaluation framework for autonomous penetration testing. *arXiv preprint arXiv:2412.01778*, 2024.

[42] Inc. NetEase Youdao. Bcembedding: Bilingual and crosslingual embedding for rag. https://github.com/netease-youdao/BCEmbedding, 2023.

[43] Nikto. Nikto web server scanner, 2024. https://github.com/sullo/nikto.

[44] Nmap. Nmap: The network mapper - free security scanner, 2024. https://nmap.org/.

[45] OWASP. Owasp-testing-guide, 2013. https://github.com/OWASP/owasp-testing-guide.

[46] Nivedita James Palatty. 83 penetration testing statistics: Key facts and figures, 2024. https://www.getastra.com/blog/security-audit/penetration-testing-statistics/.

[47] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.

[48] Derry Pratama, Naufal Suryanto, Andro Aprila Adiputra, Thi-Thu-Huong Le, Ahmada Yusril Kadiptya, Muhammad Iqbal, and Howon Kim. Cipher: Cybersecurity intelligent penetration-testing helper for ethical researcher. *Sensors*, 24(21):6878, 2024.